ÉPREUVE TECHNIQUE DE SELECTION

Architecture micro-services, IA et développement full-stack

Épreuve Pratique | Durée : 3H

Contexte métier et architecture cible

Une banque souhaite mettre en place une plateforme permettant de consulter les comptes bancaires, d'enregistrer les transactions et de publier des événements métier. Un assistant conversationnel permettra aux utilisateurs d'interroger les données depuis une interface web ou via Telegram et Discord. Le projet comprend également un volet expérimental consacré à la détection de fraudes bancaires à l'aide de modèles de machine learning développés en Python. L'architecture technique repose sur Spring Cloud pour les microservices, Spring AI pour l'assistant conversationnel, Kafka pour les échanges asynchrones et Keycloak pour l'authentification et la gestion des autorisations. L'interface web sera développée avec Angular. Les applications seront conteneurisées avec Docker, exécutées localement à l'aide de Docker Compose, et intégrées à une chaîne d'intégration et de déploiement continus basée sur GitHub Actions.

Micro-services métier

Microservice	Responsabilité	Modèle minimal
account-service	Gestion des comptes bancaires.	id: UUID customerReference: String status: ACTIVE | BLOCKED currency: String balance : double
transaction-service	Enregistrement et consultation des transactions.	id: UUID accountId: UUID amount: BigDecimal currency: String merchantCategory: String country: String createdAt: Instant

Règles métier

- Le montant doit être strictement positif et respecter le plafond configuré.
- Le compte doit exister et être actif.
- La devise de la transaction doit correspondre à celle du compte.
- Chaque nouvelle transaction nécessite la consultation du solde du compte. Après l'enregistrement de la transaction, le solde su compte est mis à jour.
- Prévoir la possibilité d'effectuer un virement d'un compte vers un autre

L'architecture comporte **deux microservices métier**. L'assistant, la gateway, la découverte et la configuration sont des composants techniques supplémentaires.

Séparation des couches

- domain/
- repository/
- service/
- web/rest/
- web/graphql/
- mcp/

Les contrôleurs REST, GraphQL et MCP doivent appeler la même couche service.

Partie A — Préparation et consignes à respecter

- Créer **un repository privé** Github ayant comme nom nom-prenom-test-adria-pfe-27
- Remplit le google sheet fourni
- Effectuer des commits sur des intervalles de 15 minutes
- Rédiger un rapport au format PDF qui contient la réponse de chaque question avec des
- Captures écrans des différents tests effectués.
- Créer un projet maven incluant un module pour chaque micro-service
- A la fin de l'épreuve, rendre votre repository public
- Rendre le code source et un rapport PDF qui contient les éléments de réponse pour les questions traitées
- Déposer le repository et le rapport dans classroom

Partie B — Conception et Architecture

1. Établir un diagramme de classes et un schéma d'architecture pour ce projet

Partie C — Développer les micro-services fonctionnels en les exposant via des connecteurs REST, GraphQL et MCP

1. Account-Service
2. Transaction-Service
3. Traiter la communication entre les deux micro-services avec OpenFeign et mettre en œuvre une solution de tolérance aux pannes basée sur Resilience4J.

Partie D — Développer les micro-services techniques

1. Spring Cloud Gateway
2. Eureka Discovery service
3. Config- service qui partage un paramètre de configuration :
    - bank.transaction.max-amount=1000

Partie E — Développer le chat bot agentique :

1. Développer un agent AI qui permet d'automatiser les opérations bancaires en utilisant un LLMs des tools connectés aux serveurs MCP des autres micro-services
2. Tester ce Chatbot avec un client REST, Client Telegram ou Discord

Partie F — Communication asynchrone avec Kafka

1. Après la persistance réussie d'une transaction, publier un événement dans `bank.transactions.created`.
2. Utiliser `accountId` comme clé du message.
3. Développer un service consumer des événements publiés dans le topic
4. Journaliser l'événement reçu, sans modifier les données bancaires.

Format minimal de l'événement

```
{
 "eventId": "UUID",
 "eventType": "TransactionCreated",
 "transactionId": "UUID",
 "accountId": "UUID",
 "amount": 250.00,
 "currency": "EUR",
 "occurredAt": "2026-01-15T10:30:00Z"
}
```

Partie G — Clients web ou Mobile (au choix)

- Développer un client React ou Angular
- Développer un client Mobile Android, IOS avec Flutter

Partie H — IA avec Python : détection de fraude

Jeu de données

Le fichier `bank_transactions.csv` contient :

```
transaction_id
account_id
amount
hour
country
merchant_category
transactions_last_24h
average_amount_last_7d
is_fraud
```

La cible `is_fraud` vaut `0` ou `1`. Les classes sont déséquilibrées. Les variables historiques sont calculées uniquement à partir des informations disponibles avant la transaction.

Travail demandé : classification binaire

1. Charger les données.
2. Afficher la proportion de fraudes et les valeurs manquantes.
3. Effectuer une séparation entraînement/test stratifiée.
4. Exclure les identifiants des variables explicatives.
5. Construire un pipeline : imputation, encodage des catégories et normalisation si nécessaire.
6. Entraîner un modèle simple : régression logistique, arbre de décision ou random forest.
7. Évaluer le modèle avec la précision et le rappel de la classe fraude, le F1-score, la matrice de confusion et l'average precision.

Le prétraitement doit être ajusté uniquement sur les données d'entraînement. L'ensemble de test ne doit pas servir à entraîner le modèle ni à sélectionner le seuil de décision.

Questions d'analyse

1. Pourquoi l'accuracy seule est-elle insuffisante ?
2. Quel compromis existe entre faux positifs et fraudes non détectées ?
3. Quel est l'effet possible d'une diminution du seuil de décision ?
4. Pourquoi privilégier une validation temporelle sur des données bancaires réelles ?

Livrer un notebook ou un script exécutable avec les résultats. Le déploiement du modèle et son intégration aux microservices ne sont pas demandés.

Partie I — DevOps

Livrables attendus

1. Créer un Dockerfile pour l'ensemble des micro-services
2. Créer un Docker-compose pour cette application
3. Créer un extrait de pipeline GitHub Actions ou GitLab CI présentant :
    - la compilation ;
    - les tests ;
    - la construction de l'image ;
    - sa publication dans un registre ;
    - une étape de déploiement.
4. Une réponse courte sur :
    - la gestion des secrets et leur distinction avec la configuration ;
    - les contrôles de démarrage, de disponibilité et de vitalité ;
    - les logs, métriques et traces distribuées ;
    - une stratégie de rollback.

Le pipeline peut être schématique. Aucun déploiement cloud réel n'est demandé.